

CSE 232 : Data Structures

LAB 06 : BINARY TREE

Due Saturday at 5:00 p.m. MST

The sixth programming assignment of this semester is to write a binary node class. This, like linked list, will not be a container. Next week, we will build the `BST` container out of the binary node we create this week.

BNode

Please place your code in `bnode.h`. It will consist of a class with three methods.

- Default Constructor
- Copy Constructor
- Move Constructor

Additionally, there will be a collection of stand-alone functions.

- **Assign:** assign (copy the values), copy (duplicate an entire tree), and swap (swap root pointers).
- **Insert:** add left node, add left value, add right node, and add right value.
- **Remove:** clear.
- **Status:** size.

Three Options

This week, there will be three options.

85 Option : Integer Base Type

This option will require you to implement all the above interfaces, but integer will be the base data-type. There are four member variables:

```
BNode * pLeft;
BNode * pRight;
BNode * pParent;
int data;
```

To complete the 85 option, please start with the following files: [232.06.Lab.85.zip](#).

100 Option : Template Base Type

This option will require you to implement all the same interfaces, but a template will be the base type.

```
template <class T>
```

There are four member variables:

```
BNode <T> * pLeft;
BNode <T> * pRight;
BNode <T> * pParent;
T data;
```

To complete the 100 option, please start with the following files: [232.06.Lab.100.zip](#).

115 Option : Allocations

This option will require the class to everything from the 100 option. Additionally, it will be necessary to perform the specified number of allocations, deletions, etc. To complete the 115 option, please start with the following files: [232.06.Lab.115.zip](#).

As with last week, the 115 option requires a precise number of allocations and deletions. In order to have more control over this, it is necessary to use an allocator. Please see Appendix I: Allocator in the text book for a detailed description of how this works. There is also a video demonstration here: [Allocators and Spys](#). This explains how the `spy` class works to count allocations, and how `std::allocator` can be use to more precisely control them.

Assignment

Please do the following:

1. Select an option: 85, 100, or 115.
2. Download the code.
3. Implement the data structure with your partner(s).
4. Report how long it took for each individual, and what was the hardest part.
5. Upload your `bnode.h` code.

Assessment

The grade for your assignment will be based on the following criteria:

	Exceptional 100%	Good 90%	Acceptable 70%	Developing 50%	Missing 0%
Code Quality 35%	The most elegant solution was found for all the methods; there is no room for improvement,	There are no errors in efficiency, maintainability, cohesion, robustness, or redundancy.	There exists one error in efficiency, maintainability, cohesion, robustness, or redundancy.	The code quality is not what is expected for a CSE 232 student.	The code is sloppy or otherwise unprofessional.
Functionality 50%	Passes all the tests	Passes 90% of unit tests	Passes 70% of unit tests	Passes 50% of unit tests	Program fails to compile or no demonstration video was provided.
Template Base Type 15%	The 100 or the 115 solution was attempted				The 85 solution was attempted
Allocations 15%	115 solution was attempted				The 85 or the 100 solution was attempted